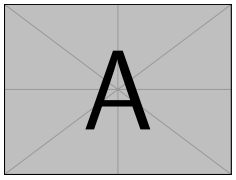
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN 1

MÔN HỌC: NHẬP MÔN TRÍ TUỆ NHÂN TẠO (CO3061)

NỘI DUNG THỰC HIỆN:

BÀI TOÁN 1: TENTS AND TREES PUZZLE

Giảng viên hướng dẫn:
Bộ môn Khoa học Máy tính

Sinh viên thực hiện:

Họ và tên	*MSSV*	*Ghi chú*
Nguyễn Huỳnh Lâm	2411841	Bài toán 1

TP. Hồ Chí Minh, Năm học 2026-2027

1.10

Mục lục

1 MÔ TẢ BÀI TOÁN VÀ MÔ HÌNH HÓA BÀI TOÁN 3
1.1 Mô tả bài toán và luật chơi . . . 3
1.1.1 Giới thiệu bài toán Tents and Trees . . . 3
1.1.2 Các quy tắc và điều kiện nghiệm . . . 3
1.1.3 Ví dụ minh họa bài toán bàn cờ 6×6 . . . 4
1.2 Mô hình hóa bài toán . . . 4
1.2.1 Trạng thái (State) . . . 4
1.2.2 Trạng thái ban đầu (Initial State) . . . 5
1.2.3 Nước đi hợp lệ (Legal Moves) . . . 5
1.2.4 Trạng thái đích (Goal State) . . . 5
1.3 Đặc tả dữ liệu đầu vào và biểu diễn trong chương trình . . . 5
1.3.1 Định dạng tệp dữ liệu đầu vào (`.txt`) . . . 5
1.3.2 Cấu trúc dữ liệu biểu diễn bàn cờ (Lớp `Board`) . . . 6

2 THIẾT KẾ MÔI TRƯỜNG VÀ GIẢI THUẬT TÌM KIẾM 7
2.1 Thiết kế môi trường mô phỏng bàn cờ (`Board`) . . . 7
2.1.1 Nguyên lý kiến trúc và phân tách trách nhiệm . . . 7
2.1.2 Cấu trúc dữ liệu và mô hình biểu diễn bộ nhớ . . . 7
2.1.3 Cơ chế kiểm soát ràng buộc và thẩm định nghiệm . . . 7
2.2 Giải thuật tìm kiếm mù: Depth-First Search (DFS) . . . 8
2.2.1 Khái niệm giải thuật DFS và áp dụng vào bài toán . . . 8
2.2.2 Cơ chế ngăn xếp đệ quy và chiến lược Quay lui (Backtracking) . . . 8
2.2.3 Tối ưu hóa không gian trạng thái bằng kỹ thuật Cắt tỉa sớm (Early Pruning) . . . 9
2.2.4 Sơ đồ quy trình thực thi giải thuật (Flowchart) . . . 9
2.2.5 Phương pháp đo lường hiệu năng thực nghiệm . . . 10
2.3 Giải thuật tìm kiếm Heuristic: Greedy Best-First Search (GBFS) . . . 11
2.3.1 Lý do lựa chọn giải thuật: Hiện tượng thoái hóa của A* và định hướng tiếp cận GBFS . . . 11
2.3.2 Mô hình hóa giải thuật Heuristic: Từ bàn cờ Board đến Kiến trúc Cây trạng thái . . . 12
2.3.3 Đề xuất phương án tối ưu kép: Con trỏ cha và Cập nhật Heuristic vi sai $\mathcal{O}(1)$ . . . 13
2.3.4 Chiến lược chọn biến MRV và cắt tỉa ràng buộc CSP . . . 14
2.3.5 Quy trình thực thi giải thuật và Lưu đồ chi tiết . . . 15

3 Giao diện mô phỏng và minh họa trực quan (Visualizer) 16
3.1 Kiến trúc ứng dụng giao diện đồ họa Tkinter (`gui_app.py`) . . . 16
3.2 Quy trình diễn hoạt từng bước (Step-by-Step Execution) . . . 17

3.3 Minh họa diễn hoạt thực nghiệm từng bước của giải thuật DFS 17
3.4 Minh họa diễn hoạt thực nghiệm từng bước của giải thuật Greedy Best-First Search 19
3.5 Nhận xét và So sánh động học giữa hai giải thuật qua giao diện đồ họa . . . 21

4 Đánh giá thực nghiệm và phân tích hiệu năng (Benchmark) 22
4.1 Thiết lập môi trường thực nghiệm và bộ dữ liệu kiểm thử 22
4.2 Bảng tổng hợp số liệu thực nghiệm 22
4.3 Phân tích so sánh hiệu năng theo các chỉ số 23
4.3.1 Không gian tìm kiếm và Số lượng nút duyệt (Nodes Explored) 23
4.3.2 Thời gian thực thi (Execution Time) 23
4.3.3 Mức độ tiêu tốn bộ nhớ đỉnh (Peak RAM) 24
4.4 Kết luận: Sự đánh đổi giữa Node, Thời gian và Bộ nhớ 25

1 MÔ TẢ BÀI TOÁN VÀ MÔ HÌNH HÓA BÀI TOÁN

1.1 Mô tả bài toán và luật chơi

1.1.1 Giới thiệu bài toán Tents and Trees

Tents and Trees là một bài toán giải đố logic dạng lưới hai chiều (Grid-based Logic Puzzle). Bài toán được định nghĩa trên một lưới ô vuông kích thước $R \times C$, gồm R hàng và C cột. Trong trạng thái ban đầu, lưới chứa một tập hợp các ô cố định có cây xanh (Tree, ký hiệu là **T**), các ô còn lại là đất trống.

Mục tiêu của bài toán là xác định vị trí đặt các túp lều (Tent, ký hiệu là **A**) vào các ô đất trống sao cho mỗi cây đều có đúng một lều tương ứng, đồng thời thỏa mãn toàn bộ các điều kiện an toàn về khoảng cách và chỉ tiêu số lượng lều ở biên của bàn cờ. Về mặt lý thuyết độ phức tạp tính toán, bài toán Tents and Trees thuộc lớp bài toán NP-đầy đủ (NP-complete).

1.1.2 Các quy tắc và điều kiện nghiệm

Một cấu hình bàn cờ được công nhận là lời giải hợp lệ khi thỏa mãn đồng thời ba quy tắc sau:

1. **Quy tắc ghép cặp kề cạnh 1-1 (1-to-1 Adjacency Constraint):** Mỗi cây phải kết nối với đúng một chiếc lều nằm tại một trong bốn ô lân cận (trên, dưới, trái, phải). Mỗi chiếc lều chỉ phục vụ cho đúng một cây duy nhất (quan hệ 1-1 giữa tập cây và tập lều). Do đó, tổng số lều trên bàn cờ bằng tổng số cây:

$$|\text{Tents}| = |\text{Trees}| \tag{1}$$

2. **Quy tắc khoảng cách an toàn (No-touching Constraint):** Hai chiếc lều bất kỳ không được phép nằm kề nhau, kể cả theo 8 hướng xung quanh (4 hướng kề cạnh và 4 hướng chéo). Với hai lều bất kỳ tại tọa độ (r_1, c_1) và (r_2, c_2), khoảng cách Chebyshev luôn lớn hơn 1:

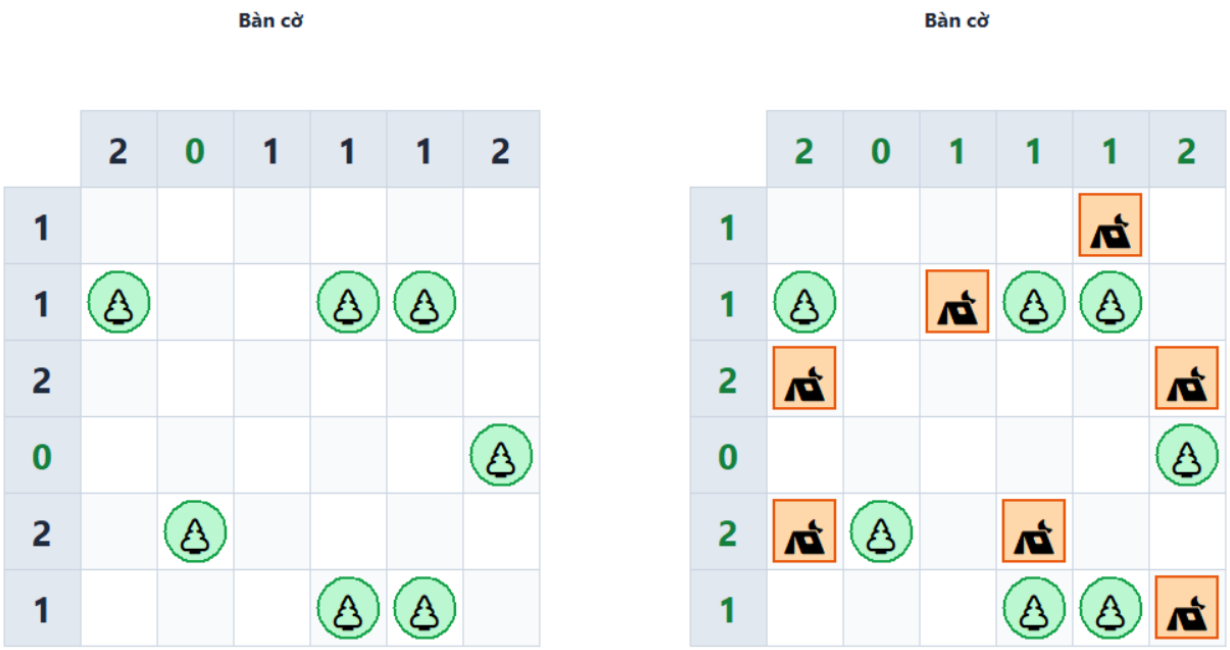
$$\max\left(|r_1 - r_2|, |c_1 - c_2|\right) > 1, \quad \forall (r_1, c_1) \neq (r_2, c_2) \in \text{Tents} \tag{2}$$

3. **Quy tắc chỉ tiêu biên theo hàng và cột (Row and Column Clues Constraint):** Số lượng lều thực tế xuất hiện trên mỗi hàng r và mỗi cột c phải bằng chính xác giá trị số chỉ dẫn (clue) nằm ở ngoài biên:

$$\sum_{c=0}^{C-1} \mathbb{I}_{((r,c)\in\text{Tents})} = \text{RowConstraint}[r], \quad \forall r \in \{0, 1, \ldots, R-1\} \tag{3}$$

$$\sum_{r=0}^{R-1} \mathbb{I}_{((r,c)\in\text{Tents})} = \text{ColConstraint}[c], \quad \forall c \in \{0, 1, \ldots, C-1\} \tag{4}$$

trong đó $\mathbb{I}_{(\cdot)}$ là hàm chỉ thị, nhận giá trị 1 nếu ô (r, c) có chứa lều và 0 nếu ngược lại.



(a) Trạng thái ban đầu (b) Trạng thái sau khi giải

Hình 1: Minh họa bàn cờ 6×6 trước và sau khi giải quyết.

1.1.3 Ví dụ minh họa bài toán bàn cờ 6×6

Hình 1 minh họa một bài toán kích thước 6×6 với 7 cây xanh tại các trạng thái trước và sau khi giải:

- **Ở trạng thái ban đầu (Hình 1a):** Lưới gồm 7 cây. Chỉ tiêu lều cho các hàng từ trên xuống dưới lần lượt là $[1, 1, 2, 0, 2, 1]$ và các cột từ trái sang phải lần lượt là $[2, 0, 1, 1, 1, 2]$. Hàng 3 và Cột 1 có chỉ tiêu bằng 0 (thể hiện bằng số 0 màu xanh đậm), quy định không được phép đặt lều tại các vị trí thuộc hàng và cột này.
- **Ở trạng thái đích sau khi giải (Hình 1b):** Toàn bộ 7 chiếc lều đã được bố trí vào các ô đất trống. Mỗi cây đều có đúng 1 lều kề cạnh, không có hai lều nào tiếp xúc 8 hướng và toàn bộ các hàng/cột đều đạt 100% chỉ tiêu (các chỉ số biên chuyển sang màu xanh lá cây).

1.2 Mô hình hóa bài toán

Bài toán được mô hình hóa theo 4 thành phần cơ bản:

1.2.1 Trạng thái (State)

Mỗi trạng thái s trong không gian tìm kiếm được biểu diễn bằng bộ hai:

$$s = \langle k, \text{tents} \rangle \tag{5}$$

Trong đó:

- $k \in \{0, 1, \ldots, N\}$: Chỉ số của cây tiếp theo đang cần xét gán lều. Danh sách N cây trên bàn cờ được đánh số cố định từ 0 đến $N-1$ theo thứ tự từ trên xuống dưới, từ trái sang phải. Khi $k = N$, toàn bộ các cây đã được duyệt xong.

- tents: Tập hợp tọa độ các ô đã được đặt lều trong cấu hình hiện tại:

$$\text{tents} = \{(r_i, c_i) \mid 0 \leq r_i < R, 0 \leq c_i < C\} \tag{6}$$

1.2.2 Trạng thái ban đầu (Initial State)

Trạng thái khởi đầu s_0 khi chưa có chiếc lều nào được đặt trên bàn cờ:

$$s_0 = \langle 0, \emptyset \rangle \tag{7}$$

1.2.3 Nước đi hợp lệ (Legal Moves)

Tại trạng thái $s = \langle k, \text{tents} \rangle$ với $k < N$, giả sử cây thứ k có tọa độ (r_T, c_T). Một nước đi hợp lệ là việc chọn đặt một chiếc lều vào ô (r, c) thỏa mãn đồng thời 4 điều kiện:

1. Tọa độ (r, c) nằm trong phạm vi bàn cờ và kề cạnh (trên, dưới, trái, phải) với cây thứ k:
$$(r, c) \in \{(r_T \pm 1, c_T), (r_T, c_T \pm 1)\} \cap ([0, R-1] \times [0, C-1]) \tag{8}$$
2. Ô (r, c) là ô trống: không chứa cây ($\text{Grid}[r][c] \neq \mathbf{T}$) và chưa có lều ($(r, c) \notin \text{tents}$).
3. Ô (r, c) không tiếp xúc với bất kỳ chiếc lều nào hiện có trong tập tents theo 8 hướng:
$$\forall (r', c') \in \text{tents} : \max(|r - r'|, |c - c'|) > 1 \tag{9}$$
4. Đặt lều tại (r, c) không làm vượt quá chỉ tiêu số lều của hàng r và cột c:
$$\text{count_row}(r, \text{tents}) + 1 \leq \text{RowConstraint}[r] \tag{10}$$
$$\text{count_col}(c, \text{tents}) + 1 \leq \text{ColConstraint}[c] \tag{11}$$

1.2.4 Trạng thái đích (Goal State)

Một trạng thái $s = \langle k, \text{tents} \rangle$ được công nhận là trạng thái đích khi thỏa mãn đồng thời 3 điều kiện:

1. Đã duyệt qua toàn bộ số cây trên bàn cờ: $k = N = |\text{Trees}|$.
2. Tổng số lều được đặt bằng đúng tổng số cây: $|\text{tents}| = N$.
3. Toàn bộ các hàng và cột đều đạt chính xác chỉ tiêu quy định:
$$\forall r \in \{0, 1, \ldots, R-1\} : \text{count_row}(r, \text{tents}) = \text{RowConstraint}[r] \tag{12}$$
$$\forall c \in \{0, 1, \ldots, C-1\} : \text{count_col}(c, \text{tents}) = \text{ColConstraint}[c] \tag{13}$$

1.3 Đặc tả dữ liệu đầu vào và biểu diễn trong chương trình

1.3.1 Định dạng tệp dữ liệu đầu vào (`.txt`)

Dữ liệu của mỗi bài toán được lưu trữ trong một tệp văn bản thuần định dạng `.txt` với cấu trúc quy định như sau:

- **Dòng 1:** Gồm 2 số nguyên R và C biểu thị số hàng và số cột của lưới.

- **Dòng 2:** Dãy gồm R số nguyên phân tách bởi dấu cách, biểu thị chỉ tiêu số lều cho từng hàng từ trên xuống dưới.
- **Dòng 3:** Dãy gồm C số nguyên phân tách bởi dấu cách, biểu thị chỉ tiêu số lều cho từng cột từ trái sang phải.
- R **dòng tiếp theo:** Ma trận kích thước $R \times C$ mô tả bản đồ ban đầu, trong đó ký tự `T` biểu thị ô có cây và ký tự `.` biểu thị ô đất trống.

  Ví dụ cấu trúc tệp dữ liệu cho bài toán minh họa 6×6 trong Hình 1:

```
6 6
1 1 2 0 2 1
2 0 1 1 1 2
. . . . . .
T . . T T .
. . . . . .
. . . . . T
. T . . . .
. . . T T .
```

1.3.2 Cấu trúc dữ liệu biểu diễn bàn cờ (Lớp `Board`)

Môi trường bài toán được đóng gói trong lớp `Board` thuộc module `src/board.py`, bao gồm các thuộc tính và phương thức:

1. Các thuộc tính lưu trữ:

- `rows`, `cols` (kiểu `int`): Kích thước số hàng và số cột của bàn cờ.
- `row_constraints`, `col_constraints` (kiểu `list[int]`): Danh sách lưu chỉ tiêu số lều của từng hàng và từng cột.
- `grid` (kiểu `list[list[str]]`): Ma trận lưu cấu hình bản đồ ban đầu.
- `trees` (kiểu `list[tuple[int, int]]`): Danh sách tọa độ các cây trên bàn cờ theo thứ tự quét dòng.
- `tents` (kiểu `set[tuple[int, int]]`): Tập hợp lưu trữ tọa độ các lều đang được đặt trong cấu hình hiện thời.

2. Các phương thức chính:

- `load_from_file(filename)`: Đọc dữ liệu từ file input, thiết lập các mảng ràng buộc và danh sách tọa độ cây `trees`.
- `get_adjacent_cells(r, c)`: Trả về danh sách các ô kề cạnh (trên, dưới, trái, phải) nằm trong giới hạn của bàn cờ.
- `has_neighbor_tent(r, c)`: Kiểm tra sự tồn tại của bất kỳ chiếc lều nào trong phạm vi 8 ô xung quanh ô (r, c).
- `can_place_tent(r, c)`: Xác định tính hợp lệ của việc đặt lều tại ô (r, c) dựa trên quy tắc ô trống, khoảng cách an toàn 8 hướng và chỉ tiêu biên hiện hành.
- `is_goal()`: Thẩm định xem cấu hình hiện tại của bàn cờ đã thỏa mãn toàn bộ các điều kiện nghiệm hay chưa.

2 THIẾT KẾ MÔI TRƯỜNG VÀ GIẢI THUẬT TÌM KIẾM

2.1 Thiết kế môi trường mô phỏng bàn cờ (`Board`)

2.1.1 Nguyên lý kiến trúc và phân tách trách nhiệm

Hệ thống được tổ chức theo mô hình phân tách độc lập giữa hai thành phần: **Môi trường bài toán (`Board`)** và **Tác tử tìm kiếm (Search Agent)**.

- Lớp `Board` đảm nhận vai trò mô phỏng trạng thái và kiểm soát luật chơi: phân tích tệp dữ liệu đầu vào, lưu trữ ma trận địa hình, quản lý các điều kiện biên và cung cấp các phương thức thẩm định tính hợp lệ của hành động.
- Tác tử tìm kiếm thực hiện các chiến lược duyệt không gian trạng thái (DFS, A*, Greedy...). Tác tử tương tác với môi trường thông qua các giao tiếp lập trình (API) do lớp `Board` cung cấp mà không can thiệp trực tiếp vào cấu trúc lưu trữ nội tại của bản đồ.

2.1.2 Cấu trúc dữ liệu và mô hình biểu diễn bộ nhớ

Các cấu trúc dữ liệu trong lớp `Board` được thiết lập nhằm tối ưu hóa chi phí thời gian của các thao tác truy vấn và cập nhật:

1. Kích thước và chỉ tiêu biên (`rows`, `cols`, `row_constraints`, `col_constraints`):

- Kích thước lưới R, C được lưu trữ bằng các biến nguyên, phục vụ kiểm tra điều kiện biên tọa độ với độ phức tạp $\mathcal{O}(1)$.
- Chỉ tiêu số lều của từng hàng và cột được lưu trữ trong hai mảng một chiều kích thước tương ứng R và C, hỗ trợ tra cứu hạn mức hàng và cột trong thời gian $\mathcal{O}(1)$.

2. Danh sách tọa độ cây (`trees`):

- Tọa độ của các ô chứa cây được trích xuất từ ma trận ban đầu và lưu trữ cố định trong danh sách:

$$\text{trees} = [(r_0, c_0), (r_1, c_1), \ldots, (r_{N-1}, c_{N-1})] \tag{14}$$

  theo thứ tự quét dòng (row-major order). Cấu trúc này cho phép giải thuật tìm kiếm truy xuất trực tiếp cây cần xét theo chỉ số tuần tự từ 0 đến $N-1$, loại bỏ thao tác duyệt quét ma trận $R \times C$ ở mỗi bước rẽ nhánh.

3. Tập hợp vị trí lều hiện thời (`tents`):

- Cấu hình các lều đã đặt trên bàn cờ được biểu diễn bằng cấu trúc tập hợp bảng băm (`set[tuple[int, int]]`).
- Cấu trúc này cho phép các thao tác bổ sung lều, hoàn tác lều và kiểm tra sự hiện diện của lều tại một tọa độ đạt độ phức tạp thời gian trung bình $\mathcal{O}(1)$, đồng thời không yêu cầu sao chép dữ liệu ma trận khi chuyển trạng thái.

2.1.3 Cơ chế kiểm soát ràng buộc và thẩm định nghiệm

Lớp `Board` hiện thực hai cấp độ kiểm tra điều kiện:

1. **Thẩm định tính hợp lệ của nước đi (`can_place_tent`):** Một vị trí đề xuất (r, c) được chấp nhận khi thỏa mãn đồng thời bốn điều kiện:
   - Tọa độ (r, c) nằm trong phạm vi bàn cờ: $0 \leq r < R$ và $0 \leq c < C$.
   - Ô (r, c) là ô trống: Grid$[r][c] \neq \mathbf{T}$ và $(r, c) \notin$ tents.
   - Không tiếp xúc với các lều hiện có theo 8 hướng (khoảng cách Chebyshev lớn hơn 1), được kiểm tra qua phương thức `has_neighbor_tent`.
   - Số lều hiện có trên hàng r và cột c chưa đạt đến giá trị chỉ tiêu biên tương ứng.
2. **Thẩm định trạng thái đích (`is_goal`):** Xác nhận tính kết thúc của bài toán khi toàn bộ N cây đã được gán lều, thỏa mãn điều kiện số lượng $|\text{tents}| = |\text{trees}|$ và số lều trên mỗi hàng, mỗi cột bằng chính xác giá trị chỉ dẫn biên.

2.2 Giải thuật tìm kiếm mù: Depth-First Search (DFS)

2.2.1 Khái niệm giải thuật DFS và áp dụng vào bài toán

Giải thuật tìm kiếm theo chiều sâu (Depth-First Search – DFS) là một giải thuật tìm kiếm mù (Uninformed Search) trên đồ thị hoặc cây tìm kiếm. Giải thuật bắt đầu từ nút gốc và tiến hành mở rộng trạng thái dọc theo từng nhánh đến độ sâu lớn nhất trước khi thực hiện quay lui. Cấu trúc quản lý các trạng thái chờ duyệt của DFS hoạt động theo nguyên tắc vào sau – ra trước (LIFO – Last In, First Out).

Khi áp dụng giải thuật DFS vào bài toán Tents and Trees, không gian tìm kiếm được mô hình hóa thành một cây tìm kiếm (Search Tree) với các đặc tính:

- **Nút gốc:** Biểu diễn trạng thái ban đầu khi chưa có chiếc lều nào được đặt trên lưới ($s_0 = \langle 0, \emptyset \rangle$).
- **Các tầng của cây:** Tầng thứ k ($0 \leq k \leq N$) tương ứng với trạng thái đã hoàn thành việc phân bổ lều cho k cây đầu tiên.
- **Các nhánh rẽ:** Nhánh chuyển tiếp từ nút tầng k sang tầng $k+1$ đại diện cho một hành động đặt lều tại một trong bốn ô lân cận trực giao của cây thứ k.
- **Độ sâu cực đại:** Chiều sâu của cây tìm kiếm cố định và bằng đúng số lượng cây N hiện diện trên bàn cờ.
- **Quy mô không gian trạng thái lý thuyết:** Với mỗi cây có tối đa 4 vị trí đặt lều kề cạnh, hệ số rẽ nhánh tối đa là $b = 4$, tạo ra không gian tìm kiếm cực đại gồm 4^N cấu hình trạng thái.

2.2.2 Cơ chế ngăn xếp đệ quy và chiến lược Quay lui (Backtracking)

Giải thuật được tổ chức dưới dạng hàm đệ quy `_backtrack(tree_index)` thông qua ngăn xếp cuộc gọi hệ thống (Call Stack):

- **Cơ chế ngăn xếp:** Mỗi tầng đệ quy lưu trữ ngữ cảnh xử lý của một cây cụ thể. Chiều sâu ngăn xếp tối đa không vượt quá N, xác định độ phức tạp không gian lưu trữ của giải thuật là $\mathcal{O}(N)$.
- **Chiến lược Quay lui (Backtracking):** Tại mỗi bước xét cây k, giải thuật duyệt qua các ô lân cận trực giao. Khi tìm thấy ô (nr, nc) thỏa mãn điều kiện `can_place_tent`, tọa độ này được thêm vào tập `tents` và hàm đệ quy được gọi cho cây kế tiếp $(k + 1)$.

Nếu nhánh con dẫn đến trạng thái không còn ô hợp lệ để gán lều cho các cây tiếp theo, giải thuật thực hiện thao tác hoàn tác: loại bỏ (nr, nc) khỏi tập `tents` để khôi phục cấu hình bàn cờ trước đó và tiếp tục kiểm tra các ô lân cận còn lại.

- **Tính bất biến trạng thái:** Quá trình thử nghiệm và hoàn tác bảo đảm tính bất biến: cấu hình lều trên bàn cờ tại mọi thời điểm luôn thỏa mãn quy tắc khoảng cách an toàn 8 hướng.

2.2.3 Tối ưu hóa không gian trạng thái bằng kỹ thuật Cắt tỉa sớm (Early Pruning)

Để giảm thiểu số lượng trạng thái cần duyệt so với quy mô lý thuyết 4^N, giải thuật tích hợp kỹ thuật Cắt tỉa sớm (Early Pruning) dựa trên tính chất của bài toán:

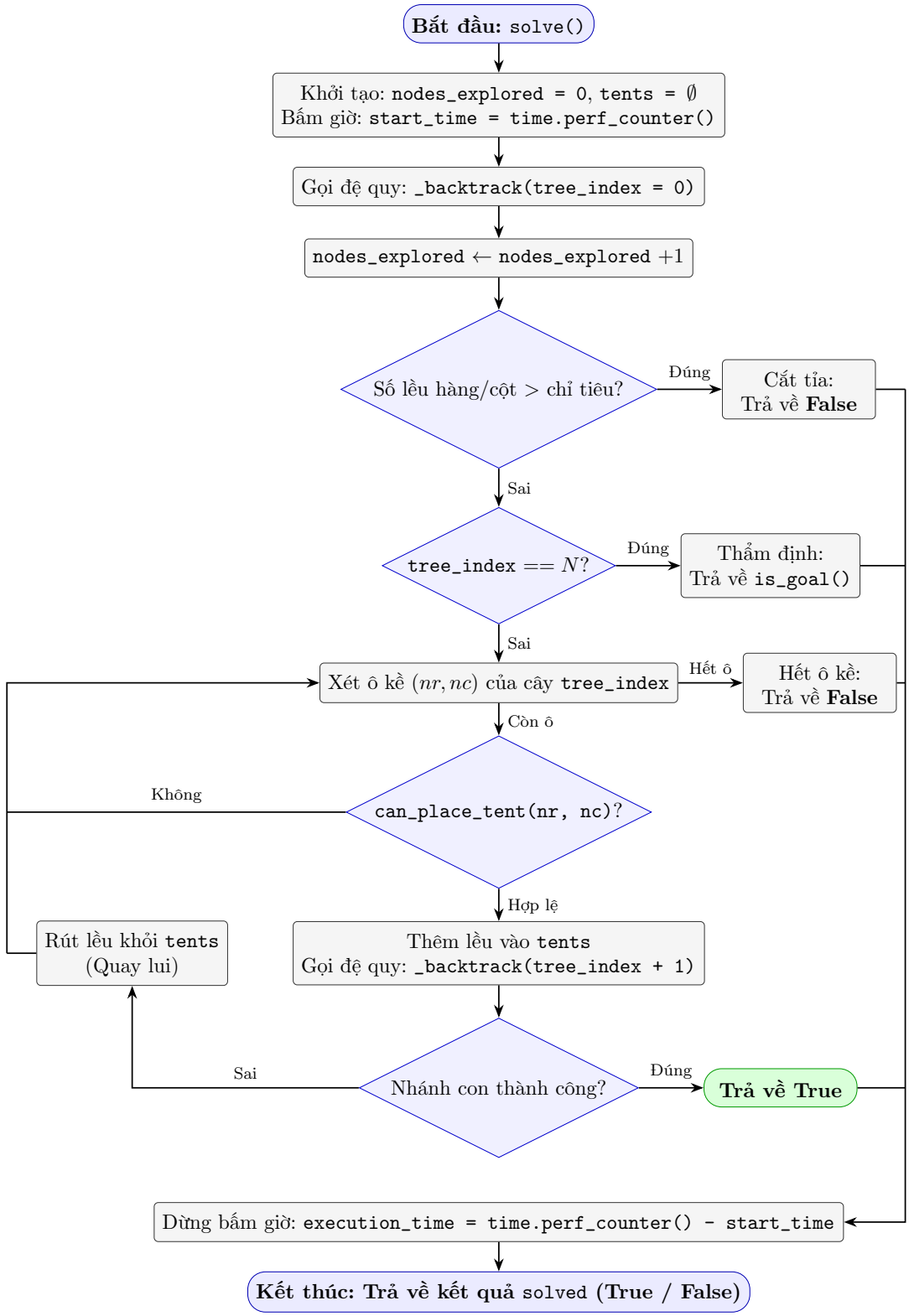
- **Tính đơn điệu của việc đặt lều:** Quá trình duyệt sâu chỉ thực hiện thao tác bổ sung lều, do đó số lượng lều trên mỗi hàng r và mỗi cột c là một hàm đơn điệu không giảm theo độ sâu của cây tìm kiếm.
- **Điều kiện cắt tỉa:** Tại mỗi bước gọi đệ quy, giải thuật kiểm tra số lượng lều thực tế của các hàng và các cột. Nếu tồn tại hàng r hoặc cột c có số lượng lều vượt quá hạn mức quy định:

$$\exists r : \text{count_row}(r) > \text{RowConstraint}[r] \quad \vee \quad \exists c : \text{count_col}(c) > \text{ColConstraint}[c] \quad (15)$$

nhánh tìm kiếm hiện tại lập tức dừng thực thi và trả về giá trị **False**. Thao tác này triệt tiêu toàn bộ cây con bên dưới nhánh vi phạm mà không cần mở rộng các cây tiếp theo.

2.2.4 Sơ đồ quy trình thực thi giải thuật (Flowchart)

Quy trình vận hành toàn diện của giải thuật DFS được mô tả thông qua sơ đồ luồng tại Hình 2. Sơ đồ thể hiện một chu trình khép kín: khởi đầu từ phương thức điều khiển chính `solve()` (thiết lập trạng thái ban đầu, kích hoạt bộ bấm giờ), chuyển tiếp sang hàm đệ quy `_backtrack()` với các chốt thẩm định cắt tỉa sớm theo chỉ tiêu biên, kiểm tra điều kiện dừng toàn cục và cơ chế thử nghiệm – hoàn tác (quay lui), cuối cùng tính toán thời gian thực thi và trả về kết quả nghiệm.



Hình 2: Sơ đồ luồng toàn diện của giải thuật DFS từ khâu khởi tạo, chu trình đệ quy kết hợp quay lui và ghi nhận hiệu năng.

2.2.5 Phương pháp đo lường hiệu năng thực nghiệm

Nhằm phục vụ phân tích định lượng hiệu năng giải thuật, hai chỉ số đo lường được thu thập trong quá trình thực nghiệm:

- **Thời gian thực thi (Execution Time):** Được đo bằng hàm `time.perf_counter()` trong thư viện chuẩn Python (độ chính xác cấp nano-giây). Để triệt tiêu sai số ngẫu nhiên do cơ chế phân bổ tiến trình của hệ điều hành, mỗi phép đo được thực hiện lặp lại 5 lần độc lập để xác định giá trị thời gian trung bình (đơn vị: mili-giây – ms).
- **Bộ nhớ tiêu tốn đỉnh (Peak Memory):** Được ghi nhận thông qua module chuẩn `tracemalloc`. Cơ chế này theo dõi lượng bộ nhớ RAM cực đại được cấp phát trong suốt chu trình thực thi hàm tìm kiếm (đơn vị: Kilobytes – KB), phản ánh dung lượng bộ nhớ thực tế cần thiết để duy trì Call Stack và các cấu trúc dữ liệu liên quan.

2.3 Giải thuật tìm kiếm Heuristic: Greedy Best-First Search (GBFS)

2.3.1 Lý do lựa chọn giải thuật: Hiện tượng thoái hóa của A* và định hướng tiếp cận GBFS

Trong giai đoạn đầu của quá trình phát triển thuật toán tìm kiếm có thông tin cho bài toán *Tents and Trees*, thuật toán kinh điển A* với hàm lượng giá chuẩn $f(n) = g(n) + h(n)$ đã được triển khai thử nghiệm. Tuy nhiên, qua quá trình đánh giá thực nghiệm và phân tích cấu trúc toán học của bài toán, thuật toán A* bộc lộ một điểm nghẽn nghiêm trọng về khả năng định hướng không gian trạng thái.

Cụ thể, trong bài toán cắm lều, mỗi hành động hợp lệ luôn tương ứng với việc ghép nối đúng một chiếc lều duy nhất cho một cây xác định. Do đó, chi phí thực tế đã tích lũy $g(n)$ phản ánh chính xác số lượng cây đã được giải quyết thành công, tức tương đương tuyệt đối với độ sâu của nút trạng thái trong cây tìm kiếm: $g(n) = \text{depth}(n)$. Đối với một bài toán có tổng cộng N cây, số lượng cây còn lại cần được phân bổ lều tại bất kỳ trạng thái n nào luôn là $N - g(n)$. Nếu hàm Heuristic $h(n)$ phản ánh khoảng cách lý tưởng đến đích thông qua số cây còn lại, hàm lượng giá tổng quát của A* sẽ có dạng:

$$f(n) = g(n) + h(n) = g(n) + [N - g(n)] = N = \text{const} \tag{16}$$

Hiện tượng hàm $f(n)$ nhận cùng một giá trị hằng số N tại mọi nhánh tìm kiếm triệt tiêu hoàn toàn khả năng phân loại mức độ ưu tiên của hàng đợi Min-Heap. Khi toàn bộ các trạng thái biên đều có điểm lượng giá ngang nhau, thuật toán A* bị thoái hóa hoàn toàn thành thuật toán Tìm kiếm theo Chiều rộng (Breadth-First Search – BFS). Hàng đợi tập biên khi đó phải lưu trữ toàn bộ các trạng thái trên cùng một mức độ sâu, dẫn đến hiện tượng bùng nổ không gian bộ nhớ theo hàm mũ và khiến hệ thống nhanh chóng cạn kiệt tài nguyên RAM.

Để khắc phục triệt để nghịch lý trên, kiến trúc tìm kiếm được chuyển đổi sang giải thuật **Greedy Best-First Search (GBFS)** với hàm lượng giá được định nghĩa trực tiếp thông qua hàm Heuristic chuyên biệt:

$$f(n) \equiv h(n) \tag{17}$$

Bằng việc lược bỏ thành phần chi phí quá khứ $g(n)$, giải thuật GBFS dồn toàn bộ năng lực đánh giá vào hàm Heuristic $h(n)$ nhằm phản ánh mức độ căng thẳng ràng buộc còn lại trên bàn cờ. Giải thuật luôn ưu tiên khai phá nhánh tìm kiếm có tiềm năng giải tỏa áp lực không gian lớn nhất, định hướng đường đi thẳng tiến tới trạng thái đích và giảm thiểu tối đa số lượng trạng thái phải mở rộng.

2.3.2 Mô hình hóa giải thuật Heuristic: Từ bàn cờ Board đến Kiến trúc Cây trạng thái

1. Sự khác biệt cốt tử trong mô hình hóa so với DFS: Mô hình bàn cờ trò chơi cùng các quy tắc không gian và vector hạn ngạch hàng/cột đã được xây dựng hoàn chỉnh trong lớp `Board` ở phần trước. Trong thuật toán tìm kiếm mù DFS, quá trình tìm kiếm được vận hành thông qua cơ chế đệ quy trên Call Stack, thao tác trực tiếp trên một đối tượng bàn cờ duy nhất bằng cách cắm thử lều và quay lui rút lều tại chỗ mà không cần khởi tạo các đối tượng nút trạng thái độc lập.

Ngược lại, giải thuật tìm kiếm Heuristic có mục tiêu tìm kiếm toàn cục, đòi hỏi hệ thống phải thường xuyên so sánh mức độ tiềm năng giữa các nhánh rẽ hoàn toàn khác nhau trên khắp không gian bài toán để quyết định nhánh nào được đi tiếp. Yêu cầu này đòi hỏi không gian tìm kiếm phải được mô hình hóa dưới dạng một **Cây tìm kiếm đồ thị (Graph Search Tree)**. Mỗi lát cắt của bàn cờ tại một bước rẽ nhánh được đóng gói thành một đối tượng nút trạng thái riêng biệt (`GreedyNode`), và toàn bộ các nút này được quản lý đồng bộ thông qua hai cấu trúc dữ liệu chủ đạo là tập biên `Open` và tập khám phá `Closed`.

2. Cơ chế điều phối tập biên Open (Min-Heap) và tập khám phá Closed (Hash Set):

- **Tập biên `Open`:** Có nhiệm vụ quản lý tất cả các nút trạng thái đã được sinh ra nhưng chưa được mở rộng, bảo đảm luôn trích xuất trạng thái có triển vọng nhất theo tiêu chuẩn tối ưu toàn cục. Tập biên được hiện thực bằng cấu trúc dữ liệu Min-Heap nhị phân thông qua module `heapq`, cho phép lấy ra phần tử nhỏ nhất trong thời gian $\mathcal{O}(1)$ và chèn phần tử mới trong $\mathcal{O}(\log|\text{Open}|)$. Thứ tự ưu tiên giữa hai nút trong Min-Heap được phân xử thông qua phương thức so sánh `__lt__` gồm 3 tầng tinh gọn:
  - *Tầng 1:* Ưu tiên nút có hàm Heuristic h nhỏ nhất.
  - *Tầng 2:* Ưu tiên nút có độ sâu `depth` lớn hơn nhằm khuyến khích giải thuật đào sâu tiến về đích.
  - *Tầng 3:* Phân xử theo thứ tự tọa độ tự nhiên của ô lều vừa cắm (hàng trên trước, cột bên trái trước) nhằm bảo đảm tính tất định 100% của giải thuật.
- **Tập khám phá `Closed`:** Đóng vai trò là một bộ lọc kiểm soát trùng lặp, ngăn chặn giải thuật mở rộng lại các cấu hình bàn cờ đã từng được phân tích thông qua những con đường khác nhau. Tập `Closed` được hiện thực dưới dạng một Bảng băm (Hash Set) lưu trữ các khóa trạng thái bất biến gồm tuple:

$$\text{Key}_{\text{Closed}} = (\text{frozenset}(\text{assigned_trees}), \text{frozenset}(\text{tents})) \tag{18}$$

  Với hàm băm tối ưu, thao tác kiểm tra sự tồn tại và bổ sung trạng thái vào tập `Closed` đạt độ phức tạp trung bình là $\mathcal{O}(1)$, triệt tiêu hoàn toàn nguy cơ lặp vòng vô tận.

3. Bản chất toán học và trực giác của hàm Heuristic $h(n)$: Hàm Heuristic $h(n)$ được thiết kế nhằm định lượng mức độ nguy cơ bế tắc và áp lực không gian trên toàn bộ bàn cờ tại trạng thái n. Ký hiệu $\mathcal{U}(n)$ là tập hợp các cây chưa được ghép nối với lều, và $\mathcal{D}(T)$ là miền giá trị hợp lệ của cây $T \in \mathcal{U}(n)$, bao gồm các ô kề 4 hướng hiện vẫn thỏa mãn tất cả các ràng buộc đặt lều (chưa bị chiếm giữ, không chạm lều khác và thuộc hàng/cột chưa vượt chỉ tiêu). Hàm Heuristic $h(n)$ được xác định theo công thức:

$$h(n) = \sum_{T \in \mathcal{U}(n)} \frac{1}{|\mathcal{D}(T)|} \tag{19}$$

Ý nghĩa vật lý của công thức trên nằm ở phép nghịch đảo kích thước miền giá trị. Một cây còn nhiều ô kề hợp lệ, ví dụ $|\mathcal{D}(T)| = 4$, chỉ đóng góp một lượng nhỏ 0.25 vào hàm mục tiêu, phản ánh rằng biến này còn nhiều sự tự do và xác suất gây tắc nghẽn là rất thấp. Ngược lại, khi các lều xung quanh thu hẹp các phương án của một cây xuống chỉ còn đúng 1 ô duy nhất ($|\mathcal{D}(T)| = 1$), cây đó sẽ đóng góp mức tối đa là 1.00 vào hàm lượng giá, phát tín hiệu cảnh báo rằng trạng thái này đang ở trong tình thế ngặt nghèo. Tổng các giá trị nghịch đảo này đóng vai trò như một thước đo ứng suất ràng buộc của toàn bộ lưới cờ. Trạng thái đích đạt được khi toàn bộ N cây đều đã được ghép lều thành công, khi đó tập $\mathcal{U}(n)$ trở thành rỗng và giá trị hàm Heuristic triệt tiêu về 0.

2.3.3 Đề xuất phương án tối ưu kép: Con trỏ cha và Cập nhật Heuristic vi sai $\mathcal{O}(1)$

1. Thiết kế ban đầu và điểm nghẽn hiệu năng nhận diện được: Trong thiết kế ban đầu, mỗi nút trạng thái được khởi tạo theo cơ chế sao chép toàn phần (Full-State Cloning), tức mỗi khi sinh ra một nút con, hệ thống phải nhân bản toàn bộ tập hợp tọa độ các lều đã đặt cùng với danh sách lịch sử đường đi. Cơ chế này tiêu tốn trung bình từ 1.0 KB đến 1.2 KB bộ nhớ cho mỗi nút, khiến việc duyệt hàng vạn trạng thái trên các bàn cờ lớn nhanh chóng làm cạn kiệt tài nguyên RAM.

Đồng thời, phương pháp đánh giá Heuristic ban đầu được thực hiện theo cách duyệt toàn cục: tại mỗi nút con mới sinh, giải thuật duyệt qua toàn bộ N cây trên bàn cờ để đếm lại số ô hợp lệ từ đầu với độ phức tạp $\mathcal{O}(N)$. Trong thực tế, một chiếc lều mới cắm ở một góc bàn cờ chỉ tác động đến một vài cây nằm sát cạnh nó, trong khi phần lớn các cây ở khoảng cách xa hoàn toàn không thay đổi số ô hợp lệ nhưng vẫn bị tính toán lại, gây lãng phí năng lực xử lý của CPU.

2. Cải tiến 1: Tối ưu hóa cấu trúc dữ liệu bằng Con trỏ cha (Structural Sharing): Nhằm giải quyết điểm nghẽn bộ nhớ, phương án chia sẻ cấu trúc (Structural Sharing / Parent Pointer) được triển khai. Thay vì sao chép toàn bộ dữ liệu bàn cờ, mỗi nút `GreedyNode` chỉ lưu trữ thông tin vi sai của nước đi vừa thực hiện gồm `action = (tree_index, r, c)` cùng một con trỏ tham chiếu `parent` trỏ ngược về nút cha sinh ra nó, kết hợp với hai giá trị vô hướng là độ sâu `depth` và chi phí Heuristic `h`. Toàn bộ chuỗi lịch sử quá khứ được kế thừa tự nhiên thông qua chuỗi con trỏ cha mà không tốn thêm bộ nhớ dư thừa.

Cải tiến này giúp kích thước của một nút trạng thái giảm mạnh từ 1.1 KB xuống chỉ còn **116 bytes**, tương đương mức tiết kiệm gần 90% dung lượng RAM. Khi giải thuật chạm đích, lời giải hoàn chỉnh được tái hiện chuẩn xác thông qua việc lần ngược chuỗi con trỏ từ nút đích về nút gốc với độ phức tạp thời gian chỉ là $\mathcal{O}(N)$, không gây độ trễ tính toán.

3. Cải tiến 2: Tối ưu hóa tính toán bằng Heuristic vi sai cục bộ $\mathcal{O}(1)$ (Incremental Update): Nhằm loại bỏ chi phí tính toán toàn cục dư thừa, phương án cập nhật Heuristic vi sai được ứng dụng. Thuật toán chỉ thực hiện phép quét toàn diện đúng một lần duy nhất tại nút gốc n_0 trên bàn cờ ban đầu để xác lập giá trị Heuristic nền tảng:

$$h_0 = \sum_{T \in \mathcal{U}_0} \frac{1}{|\mathcal{D}(T)|} \tag{20}$$

Tại các bước mở rộng tiếp theo, khi một chiếc lều được đặt thử tại tọa độ (r, c) cho cây T_{assigned}, giải thuật không duyệt lại toàn bộ N cây mà chỉ xác định tập hợp các cây thực sự bị ảnh hưởng, ký hiệu là Affected. Tập Affected bao gồm các cây chưa gán nằm trong bán kính khoảng cách Chebyshev $\max(|r - r_T|, |c - c_T|) \leq 2$ hoặc các cây nằm kề hàng/cột

vừa đạt đủ chỉ tiêu lều. Giá trị Heuristic mới của nút con được cập nhật trực tiếp từ giá trị Heuristic của nút cha thông qua công thức vi sai:

$$h_{\text{mới}} = h_{\text{cũ}} - \frac{1}{|\mathcal{D}(T_{\text{assigned}})|} + \sum_{T \in \text{Affected}} \left(\frac{1}{|\mathcal{D}_{\text{mới}}(T)|} - \frac{1}{|\mathcal{D}_{\text{cũ}}(T)|} \right) \tag{21}$$

Do kích thước của tập Affected luôn là một hằng số nhỏ (thông thường từ 2 đến 3 cây) bất kể quy mô bàn cờ, công thức vi sai trên đã đưa độ phức tạp của bước đánh giá Heuristic từ $\mathcal{O}(N)$ về mức $\mathcal{O}(1)$, giúp giải thuật vận hành với tốc độ vượt trội.

2.3.4 Chiến lược chọn biến MRV và cắt tỉa ràng buộc CSP

1. Chiến lược chọn biến MRV (Minimum Remaining Values – Fail-First Principle): Trong các bài toán thỏa mãn ràng buộc, thứ tự chọn biến để gán giá trị đóng vai trò quyết định đến kích thước của toàn bộ cây tìm kiếm. Thay vì duyệt tuần tự các cây theo chỉ mục cố định, giải thuật áp dụng chiến lược MRV để mang lại ba lợi ích cốt lõi:

- **Cơ chế phát hiện ngõ cụt tại chỗ (Dead-end Pruning):** Dựa trên nguyên lý thất bại sớm, nếu trong quá trình đánh giá, giải thuật phát hiện có bất kỳ cây nào đã bị các lều trước đó triệt tiêu toàn bộ miền giá trị hợp lệ ($|\mathcal{D}(T)| = 0$), trạng thái hiện tại được xác định chắc chắn là một ngõ cụt không thể dẫn tới lời giải. Khi đó, giải thuật lập tức cắt tỉa toàn bộ nhánh tìm kiếm này mà không sinh bất kỳ nút con nào vào tập biên `Open`.
- **Thu hẹp hệ số phân nhánh:** Bằng việc luôn ưu tiên mở rộng cây có số lượng ô kề khả dĩ ít nhất, hệ số phân nhánh tại mỗi bước được ép xuống mức cực tiểu. Đối với các cây chỉ còn duy nhất 1 ô hợp lệ, hệ số phân nhánh đạt mức tối ưu $b = 1$, biến bước đi này thành một lựa chọn cưỡng bức tất yếu và ngăn chặn hiện tượng bùng nổ tổ hợp.
- **Quy tắc phân xử hòa tất định (Deterministic Tie-breaking):** Trong trường hợp tồn tại nhiều cây chưa gán cùng đạt số ô hợp lệ cực tiểu ($|\mathcal{D}(T_i)| = |\mathcal{D}(T_j)| = \min$), giải thuật ưu tiên cây xuất hiện trước theo thứ tự đọc tự nhiên của lưới ô vuông: quét từ trên xuống dưới và từ trái sang phải (chỉ mục index nhỏ hơn). Tương tự, khi đưa các trạng thái con vào tập biên `Open`, nếu hai trạng thái có cùng giá trị $h(n)$, hàng đợi ưu tiên phân xử theo độ sâu depth lớn hơn (đã cắm được nhiều cây hơn), và cuối cùng là tọa độ ô lều (r, c) theo thứ tự quét tự nhiên để đảm bảo tính tất định 100%.

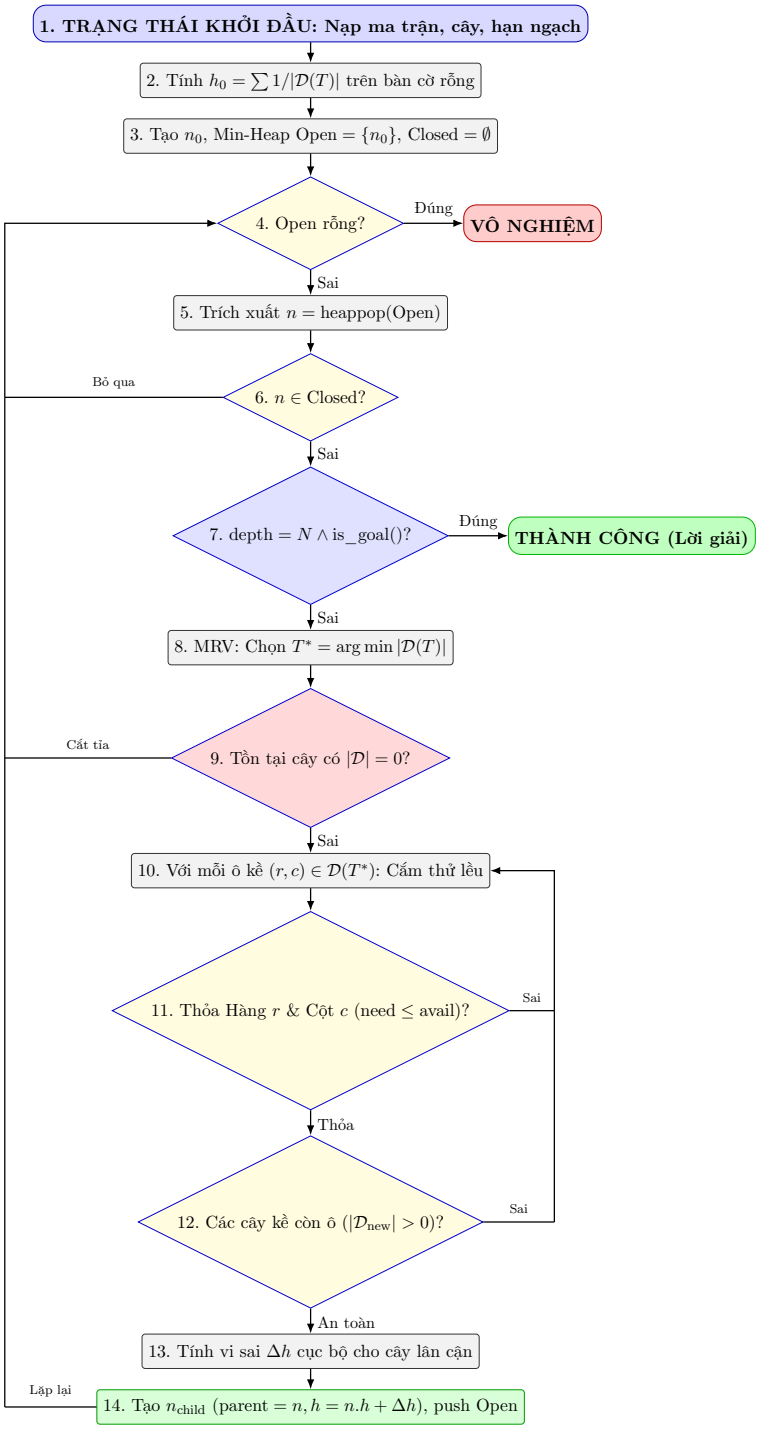
2. Kỹ thuật cắt tỉa theo sức chứa hàng và cột (Row/Column Capacity Pruning): Bên cạnh ràng buộc cục bộ giữa lều và cây, bài toán còn phải thỏa mãn các ràng buộc toàn cục về số lượng lều trên mỗi hàng và mỗi cột. Hàm `_is_row_col_satisfiable` tiến hành kiểm tra điều kiện cần của bài toán CSP tại hàng r và cột c nơi lều mới dự kiến được đặt. Gọi quota là chỉ tiêu lều của đường thẳng, placed là số lều hiện hữu sau khi đặt thử, và avail là số ô trống còn khả năng hợp lệ trên đường thẳng đó:

$$\text{need} = \text{quota} - \text{placed} \tag{22}$$

Nếu số ô khả dĩ còn lại nhỏ hơn nhu cầu cần cắm (avail $<$ need), điều đó chứng minh rằng dù các bước tiếp theo có đặt lều ở bất kỳ vị trí nào, hàng hoặc cột này cũng không bao giờ có thể đạt đủ chỉ tiêu quy định. Nhánh tìm kiếm này lập tức bị cắt tỉa trước khi nút con được tạo ra.

2.3.5 Quy trình thực thi giải thuật và Lưu đồ chi tiết

Quy trình vận hành toàn diện của giải thuật Greedy Best-First Search được mô tả thông qua sơ đồ luồng tại Hình 3. Sơ đồ thể hiện một chu trình khép kín: khởi đầu từ việc thiết lập trạng thái ban đầu, nạp bàn cờ từ dữ liệu đầu vào, tính toán Heuristic gốc h_0 và khởi tạo hàng đợi ưu tiên Min-Heap (`Open`), sau đó bước vào chu trình lặp trích xuất và mở rộng trạng thái kết hợp kỹ thuật cắt tỉa ngõ cụt theo MRV và kiểm tra sức chứa hàng/cột cho đến khi tìm thấy lời giải hợp lệ hoặc chứng minh bài toán vô nghiệm.



Hình 3: Lưu đồ toàn diện quy trình thực thi giải thuật Greedy Best-First Search từ trạng thái khởi tạo đến đích.

3 GIAO DIỆN MÔ PHỎNG VÀ MINH HỌA TRỰC QUAN (VISUALIZER)

3.1 Kiến trúc ứng dụng giao diện đồ họa Tkinter (`gui_app.py`)

Nhằm phục vụ việc quan sát động học và kiểm chứng trực quan quá trình tìm kiếm không gian trạng thái của các thuật toán, một ứng dụng giao diện đồ họa (GUI) hoàn chỉnh đã được phát triển trong tệp `problem_1_tents/gui_app.py` bằng thư viện chuẩn `tkinter` của Python (không phụ thuộc vào thư viện bên ngoài). Giao diện tổng thể được thể hiện tại Hình 4.



Hình 4: Tổng quan giao diện đồ họa Tkinter của ứng dụng Tents and Trees (`gui_app.py`) với bàn cờ trạng thái, các bảng điều khiển tác vụ, điều hướng diễn hoạt và bảng theo dõi benchmark trực tiếp.

Giao diện được phân bổ khoa học thành các khối chức năng chính:

- **Vùng hiển thị bàn cờ (Board Canvas):** Vẽ động lưới ô vuông với các biểu tượng cây xanh, lều trại và các ô cỏ. Các chỉ tiêu biên ở hàng và cột đổi màu trạng thái theo thời gian thực: hiển thị màu đỏ khi chưa đủ lều, màu xanh lá cây khi đạt đúng số lượng, và màu xanh đậm tại các hàng/cột cấm cắm lều (chỉ tiêu bằng 0).
- **Bảng điều khiển tác vụ (Control Panel):** Cho phép tùy chọn tệp bài toán đầu vào (`.txt`), chuyển đổi linh hoạt giữa giải thuật tìm kiếm mù (DFS) và tìm kiếm Heuristic (Greedy Best-First Search).
- **Cơ chế điều khiển diễn hoạt Step-by-Step:** Cung cấp các chế độ điều hướng gồm Chạy tự động (*Auto Play*) với thanh trượt tốc độ từ 100 *ms* đến 1000 *ms*, và chế độ nhảy từng bước (*Bước sau* và *Bước trước*) dựa trên lịch sử `step_history`.
- **Bảng thống kê hiệu năng trực tiếp (Live Benchmark Stats):** Cập nhật liên tục số lượng bước thử, số trạng thái đã mở rộng (Nodes Explored), thời gian giải (ms) và trạng thái thẩm định nghiệm (`is_goal`).

3.2 Quy trình diễn hoạt từng bước (Step-by-Step Execution)

Trong quá trình tìm kiếm, mỗi hành động được bộ giải thuật ghi nhận vào lịch sử dưới dạng các bộ dữ liệu vi sai: hành động cắm thử lều (`place`) hoặc hành động hoàn tác rút lều (`remove`). Khi người dùng nhấn nút điều hướng hoặc kích hoạt Auto Play, giao diện sẽ cập nhật trạng thái bàn cờ tương ứng:

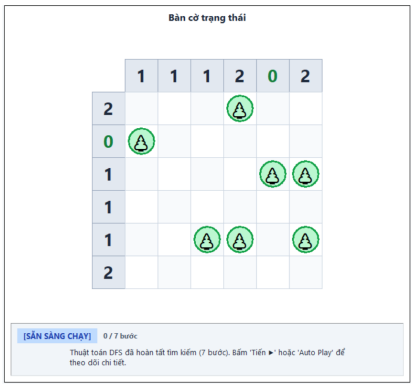
- **Đối với giải thuật DFS:** Minh họa rõ nét quá trình thử sai sâu dọc theo từng cây. Khi gặp xung đột khoảng cách hoặc vượt quota biên, giao diện thể hiện trực quan bước quay lui: rút lều vừa cắm và lùi lại cây trước đó để thử phương án khác.
- **Đối với giải thuật Greedy Best-First Search:** Thể hiện khả năng định hướng dứt khoát của hàm Heuristic và chiến lược chọn biến MRV. Thuật toán lựa chọn cây có ít ô hợp lệ nhất và cắm lều tuần tự tiến thẳng về trạng thái đích mà hầu như không cần quay lui.

3.3 Minh họa diễn hoạt thực nghiệm từng bước của giải thuật DFS

Chuỗi 8 bước diễn hoạt trực quan của giải thuật DFS trên bài toán 6×6 (7 cây xanh) được ghi nhận trực tiếp từ giao diện đồ họa Tkinter (Hình 5 và Hình 5). Toàn bộ quy trình diễn giải chi tiết bao gồm:

1. **Trạng thái ban đầu (Bước 0 - Hình 5a):** Lưới ô vuông 6×6 gồm 7 cây xanh. Các chỉ số chỉ tiêu biên chưa thỏa mãn hiển thị màu xanh đậm; các hàng và cột có chỉ tiêu 0 (Hàng 1, Cột 4) được nhận diện để cấm cắm lều. Hộp trạng thái hiển thị thông báo sẵn sàng chạy (0/7 bước).
2. **Quá trình thử cắm lều tuần tự (Bước $1-6$):** DFS duyệt tuần tự theo thứ tự tĩnh của danh sách cây ban đầu:
   - *Bước 1 (Hình 5b):* Duyệt Cây #0 tại $(0, 3)$, cắm lều tại ô kề bên trái $(0, 2)$. Đường nối nét đứt màu cam xác nhận quan hệ ghép cặp 1-1.
   - *Bước 2 (Hình 5c):* Duyệt Cây #1 tại $(1, 0)$, cắm lều tại ô phía trên $(0, 0)$. Hàng 0 đạt đủ 2 lều, chỉ số biên Hàng 0 chuyển sang màu xanh lá cây thỏa mãn.
   - *Bước 3 (Hình 5d):* Duyệt Cây #2 tại $(2, 4)$, cắm lều tại ô bên trái $(2, 3)$.
   - *Bước 4 (Hình 5e):* Duyệt Cây #3 tại $(2, 5)$, cắm lều tại ô phía dưới $(3, 5)$.
   - *Bước 5 (Hình 5f):* Duyệt Cây #4 tại $(4, 2)$, cắm lều tại ô bên trái $(4, 1)$.
   - *Bước 6 (Hình 5g):* Duyệt Cây #5 tại $(4, 3)$, cắm lều tại ô phía dưới $(5, 3)$.

   Tại mỗi bước, cơ chế `can_place_tent()` tự động khóa 8 ô lân cận xung quanh lều vừa cắm, ngăn chặn vi phạm khoảng cách an toàn.
3. **Đạt trạng thái đích (Bước 7 - Hình 5h):** Chiếc lều cuối cùng được bố trí tại $(5, 5)$ cho Cây #6 tại $(4, 5)$. Toàn bộ 7 cây đã được phân bổ đúng 1 lều tương ứng, không có hai lều nào tiếp xúc 8 hướng và 100% chỉ tiêu biên hàng/cột đều đạt đúng hạn ngạch (tất cả các số biên đều chuyển sang màu xanh lá cây đậm). Hàm `is_goal()` trả về **True**, hộp trạng thái chuyển sang thông báo [**HOÀN THÀNH**] và thuật toán hoàn tất thành công.



(a) Bước 0: Trạng thái ban đầu (Sẵn sàng chạy)



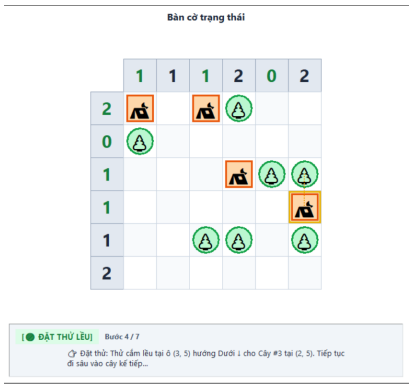
(b) Bước 1: Cây #0 tại $(0, 3) \rightarrow$ Cắm lều $(0, 2)$



(c) Bước 2: Cây #1 tại $(1, 0) \rightarrow$ Cắm lều $(0, 0)$



(d) Bước 3: Cây #2 tại $(2, 4) \rightarrow$ Cắm lều $(2, 3)$



(e) Bước 4: Cây #3 tại $(2, 5) \rightarrow$ Cắm lều $(3, 5)$

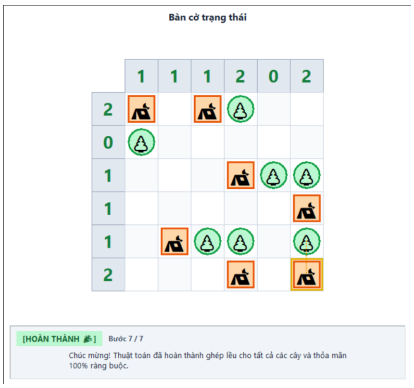


(f) Bước 5: Cây #4 tại $(4, 2) \rightarrow$ Cắm lều $(4, 1)$

Hình 5: Diễn hoạt trực quan chuỗi từng bước giải bài toán 6×6 của giải thuật DFS (Bước 0 đến Bước 5).



(g) Bước 6: Cây #5 tại $(4, 3) \rightarrow$ Cắm lều $(5, 3)$



(h) Bước 7: Cây #6 tại $(4, 5) \rightarrow$ Lều $(5, 5)$ (Hoàn thành)

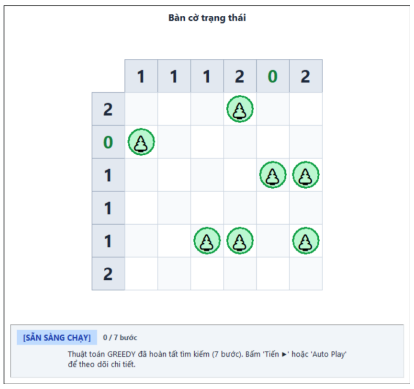
Hình 5: Diễn hoạt trực quan chuỗi từng bước giải bài toán 6×6 của giải thuật DFS (tiếp theo: Bước 6 và Bước 7, Hoàn thành).

3.4 Minh họa diễn hoạt thực nghiệm từng bước của giải thuật Greedy Best-First Search

Chuỗi 8 bước diễn hoạt trực quan của giải thuật Greedy Best-First Search trên cùng bài toán 6×6 (7 cây xanh) được ghi nhận từ giao diện Tkinter (Hình 6 và Hình 6). Toàn bộ quy trình diễn giải chi tiết bao gồm:

1. **Trạng thái ban đầu (Bước 0 - Hình 6a):** Khởi tạo hàng đợi Min-Heap (`Open`) chứa trạng thái rỗng ban đầu với giá trị Heuristic gốc h_0.
2. **Cơ chế chọn biến MRV và trích xuất** $\min h$ **(Bước 1 – 6):** Tại mỗi bước lặp, bộ giải thuật trích xuất trạng thái có giá trị h nhỏ nhất từ hàng đợi ưu tiên:
   - *Bước 1 (Hình 6b) – Chọn biến khởi đầu theo chiến lược MRV:* Khác với DFS duyệt tuần tự theo danh sách tĩnh, Greedy đánh giá số ô khả dĩ xung quanh từng cây và ưu tiên chọn cây có thể đặt được ít lều nhất (độ tự do hẹp nhất). Tại trạng thái đầu, do Hàng 1 và Cột 4 đều có chỉ tiêu bằng 0, Cây #0 tại $(0, 3)$ chỉ có thể đặt được duy nhất 1 lều tại ô $(0, 2)$ ($|\mathcal{D}(T_0)| = 1$). Theo nguyên lý Fail-First, Greedy ưu tiên cắm lều ngay tại $(0, 2)$ cho Cây #0 với giá trị $\min h = 4.50$ trích xuất từ tập `Open`.
   - *Bước 2 (Hình 6c) – Bước nhảy thông minh của chiến lược MRV:* Thay vì duyệt tuần tự sang Cây #1 như DFS, giải thuật Greedy đánh giá kích thước miền giá trị $|\mathcal{D}(T)|$ của các cây còn lại và phát hiện Cây #2 tại $(2, 4)$ chỉ còn đúng 1 ô kề hợp lệ duy nhất là $(2, 3)$. Áp dụng nguyên lý Fail-First, Greedy ưu tiên cắm lều ngay tại $(2, 3)$ cho Cây #2 để triệt tiêu nguy cơ bế tắc sớm nhất.
   - *Bước 3 (Hình 6d):* Sau khi giải tỏa điểm nghẽn tại Cây #2, giải thuật trích xuất nút có $\min h = 3.50$ và quay lại cắm lều tại $(0, 0)$ cho Cây #1 tại $(1, 0)$.
   - *Bước 4 (Hình 6e):* Trích xuất nút có $\min h = 3.00$, cắm lều tại $(3, 5)$ cho Cây #3 tại $(2, 5)$.
   - *Bước 5 (Hình 6f):* Trích xuất nút có $\min h = 2.00$, cắm lều tại $(4, 1)$ cho Cây #4 tại $(4, 2)$.
   - *Bước 6 (Hình 6g):* Trích xuất nút có $\min h = 1.00$, cắm lều tại $(5, 3)$ cho Cây #5 tại $(4, 3)$.
3. **Cán đích hoàn thành (Bước 7 - Hình 6h):** Lều cuối cùng được bố trí tại $(5, 5)$

cho Cây #6. Toàn bộ 100% chỉ tiêu biên hàng và cột đều được thỏa mãn đồng thời. Hàm `is_goal()` trả về **True** và giao diện đổi sang thông báo [**HOÀN THÀNH**].



(a) Bước 0: Khởi tạo trạng thái ban đầu (Hàng đợi Min-Heap sẵn sàng chạy)



(b) Bước 1: MRV chọn Cây #0 (chỉ 1 ô hợp lệ) $\rightarrow$ Trích xuất $\min h = 4.50$, cắm lều $(0, 2)$



(c) Bước 2: MRV ưu tiên Cây #2 $\rightarrow$ Cắm lều $(2, 3)$



(d) Bước 3: Min $h = 3.50$, Cây #1 $\rightarrow$ Cắm lều $(0, 0)$



(e) Bước 4: Min $h = 3.00$, Cây #3 $\rightarrow$ Cắm lều $(3, 5)$



(f) Bước 5: Min $h = 2.00$, Cây #4 $\rightarrow$ Cắm lều $(4, 1)$

Hình 6: Diễn hoạt trực quan chuỗi từng bước giải bài toán 6×6 của giải thuật Greedy Best-First Search (Bước 0 đến Bước 5).



(g) Bước 6: Min $h = 1.00$, Cây #5 → Cắm lều $(5, 3)$



(h) Bước 7: Cây #6 → Lều $(5, 5)$ (Hoàn thành)

Hình 6: Diễn hoạt trực quan chuỗi từng bước giải bài toán 6×6 của giải thuật Greedy Best-First Search (tiếp theo: Bước 6 và Bước 7, Hoàn thành).

3.5 Nhận xét và So sánh động học giữa hai giải thuật qua giao diện đồ họa

Qua việc quan sát trực tiếp chuỗi diễn hoạt từng bước trên ứng dụng Tkinter, nhóm nghiên cứu rút ra các so sánh thực nghiệm sâu sắc về hành vi động học của hai thuật toán:

- **Về thứ tự khám phá không gian (Static vs. Dynamic Ordering):** DFS bắt buộc phải tuân thủ thứ tự tĩnh cố định của danh sách cây ban đầu (Cây #0 → #1 → #2 → #3 ...), do đó dễ rơi vào bẫy thử sai diện rộng khi gặp các cây có ràng buộc ngặt nghèo ở cuối danh sách. Ngược lại, Greedy Best-First Search kết hợp chiến lược chọn biến MRV luôn linh hoạt nhảy đến cây có nguy cơ bế tắc cao nhất để xử lý trước (Cây #0 → #2 → #1 → #3 ...), giúp triệt tiêu nguy cơ xung đột ngay từ giai đoạn đầu.
- **Về tần suất quay lui và độ hội tụ (Convergence Backtracking):** Trên bài toán 6×6, cả hai giải thuật đều hội tụ về đích với đúng 7 bước cắm lều. Tuy nhiên, trên các bài kiểm thử quy mô lớn (10×10 và 15×15), giao diện thể hiện rõ các nhịp quay lui rút lều liên tục của DFS khi vướng vào các ô vi phạm ở nhánh sâu, trong khi Greedy di chuyển thẳng tắp về đích nhờ sự dẫn dắt chính xác của hàm Heuristic nghịch đảo độ tự do.
- **Tính trực quan hóa các ràng buộc bài toán:** Giao diện Tkinter đã thể hiện trọn vẹn và sinh động các quy tắc của trò chơi: các đường gióng liên kết màu cam biểu thị quan hệ ghép cặp 1-1, cơ chế tự động khóa 8 ô xung quanh lều bảo đảm khoảng cách an toàn, và sự đổi màu thời gian thực của các chỉ số hạn ngạch biên giúp người dùng dễ dàng kiểm chứng tính đúng đắn của lời giải.

4 ĐÁNH GIÁ THỰC NGHIỆM VÀ PHÂN TÍCH HIỆU NĂNG (BENCHMARK)

4.1 Thiết lập môi trường thực nghiệm và bộ dữ liệu kiểm thử

Để đánh giá toàn diện năng lực giải quyết bài toán và kiểm chứng các cải tiến giải thuật đã đề xuất, thực nghiệm đo đạc được triển khai trên bộ 9 bài toán kiểm thử (Test Cases) được thu thập từ nguồn dữ liệu chuẩn *Puzzle-Tents*, trải dài từ kích thước nhỏ (6×6) đến các cấu hình cực lớn và phức tạp (20×20 với 80 cây).

Cả hai giải thuật: Tìm kiếm mù (Blind Search: DFS Backtracking) và Tìm kiếm có thông tin (Heuristic Search: Greedy Best-First Search kết hợp MRV, Structural Sharing và Incremental Heuristic Update) đều được đo đạc lặp lại độc lập 3 lần để lấy giá trị trung vị (median). Các chỉ số đo lường bao gồm:

- **Thời gian thực thi (Execution Time):** Đo bằng hàm `time.perf_counter()` (đơn vị: mili-giây – ms).
- **Số lượng trạng thái mở rộng (Nodes Explored):** Đếm số nút trạng thái thực tế được trích xuất và phân tích.
- **Bộ nhớ RAM tiêu tốn đỉnh (Peak RAM):** Đo bằng module chuẩn `tracemalloc` của Python (đơn vị: Kilobytes – KB).

4.2 Bảng tổng hợp số liệu thực nghiệm

Bảng 1 tổng hợp kết quả đo đạc thực nghiệm của hai giải thuật trên toàn bộ 9 bài toán kiểm thử.

Bảng 1: Bảng tổng hợp số liệu thực nghiệm so sánh hiệu năng giữa DFS và Greedy.

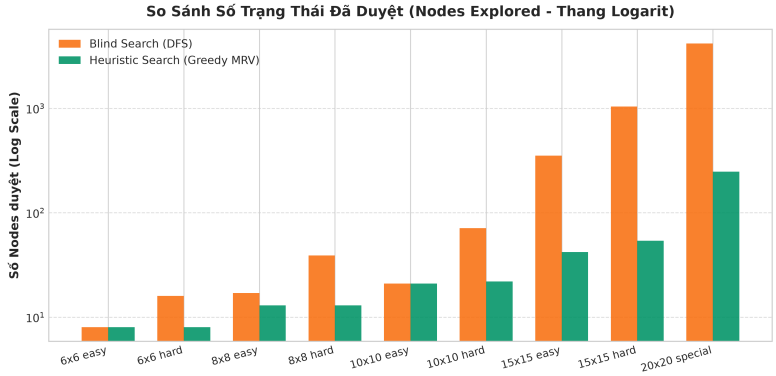
Bài kiểm thử	**Quy mô**	**Thời gian (ms)**		**Số Nodes duyệt**		**RAM đỉnh (KB)**	
(Input File)	**Lưới (N cây)**	**DFS**	**Greedy**	**DFS**	**Greedy**	**DFS**	**Greedy**
6x6 easy	6×6 (7 cây)	0.333	2.588	8	8	5.0	15.9
6x6 hard	6×6 (7 cây)	0.906	2.685	16	8	4.1	12.6
8x8 easy	8×8 (12 cây)	0.842	4.377	17	13	5.3	26.3
8x8 hard	8×8 (12 cây)	1.842	6.366	39	13	8.2	23.4
10x10 easy	10×10 (20 cây)	1.192	11.132	21	21	6.5	59.8
10x10 hard	10×10 (20 cây)	4.138	14.011	71	22	11.5	50.4
15x15 easy	15×15 (45 cây)	32.530	42.558	352	42	103.4	133.1
15x15 hard	15×15 (45 cây)	92.560	65.760	1,041	54	51.4	168.8
20x20 special	20×20 (80 cây)	491.328	2,598.222	4,174	248	1,095.1	1,457.8

4.3 Phân tích so sánh hiệu năng theo các chỉ số

4.3.1 Không gian tìm kiếm và Số lượng nút duyệt (Nodes Explored)

15x15 easy 15×15 (45 cây) 32.530 42.558 352 42 103.4 133.1

Đặc biệt tại bài toán cực đại 20×20 với 80 cây, DFS phải duyệt qua 4.174 trạng thái với ma trận quay lui phức tạp, trong khi Greedy chỉ cần khảo sát 248 trạng thái, thu hẹp đến gần 95% số nút cần kiểm tra. Điều này chứng minh hàm Heuristic nghịch đảo độ tự do kết hợp nguyên lý Fail-First của MRV đã định hướng con đường tìm kiếm đi thẳng về đích với độ chính xác cao.



Hình 7: Biểu đồ so sánh số lượng trạng thái mở rộng (Nodes Explored) giữa DFS và Greedy qua 9 bài kiểm thử.

4.3.2 Thời gian thực thi (Execution Time)

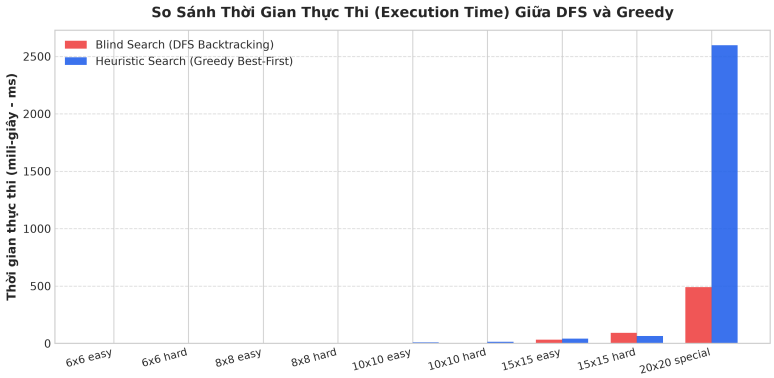
Về thời gian thực thi (Hình 8), ở các lưới kích thước nhỏ từ 6×6 đến 10×10, do không gian tìm kiếm còn hạn hẹp, cơ chế đệ quy thuần túy của DFS có thời gian chạy rất ngắn (dưới 4 ms) nhờ không phải chi trả chi phí quản lý hàng đợi ưu tiên `heapq` và các phép toán tập hợp trong Python.

Tuy nhiên, khi chuyển sang lưới 15×15 với cấu hình phức tạp (15x15 hard), điểm cân bằng hiệu năng lập tức đảo chiều nghiêng về phía Heuristic Search: Greedy hoàn tất trong khoảng 65.8 ms

$$,nhanhhnrrtsovimc$$

92.6 ms

của DFS. Sự cách biệt này phản ánh quy luật kinh điển của trí tuệ nhân tạo: khi bài toán có độ sâu lớn và nhiều nhánh rẽ, chi phí tính toán Heuristic trên mỗi nút hoàn toàn được bù đắp bởi sự sụt giảm hàm mũ của số lượng nút cần mở rộng.Riêng tại lưới 20×20, mặc dù số node của Greedy ít hơn gần 17 lần so với DFS, thời gian thực thi của Greedy lại lớn hơn do việc cập nhật Heuristic vi sai và kiểm tra ràng buộc CSP trên 80 cây trong môi trường thông dịch Python tạo ra độ trễ tính toán trên mỗi node cao hơn thao tác thử-sai đơn thuần của DFS.

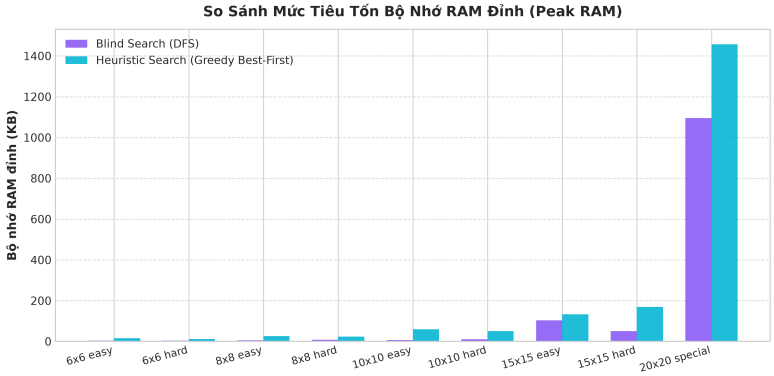


Hình 8: Biểu đồ so sánh thời gian thực thi (Execution Time) giữa DFS và Greedy qua 9 bài kiểm thử.

4.3.3 Mức độ tiêu tốn bộ nhớ đỉnh (Peak RAM)

Về mức độ tiêu tốn bộ nhớ (Hình 9), DFS thể hiện đúng đặc tính lý thuyết của tìm kiếm theo chiều sâu khi chỉ duy trì một đường đi đơn lẻ trên ngăn xếp đệ quy với độ phức tạp không gian tuyến tính $\mathcal{O}(d)$, giúp lượng RAM tiêu thụ ở các bài test nhỏ luôn ở mức tối thiểu, dao động từ 4 KB đến 51 KB.

Ngược lại, giải thuật tìm kiếm đồ thị Greedy phải lưu trữ đồng thời cả tập biên `Open` và tập đóng `Closed`, tiềm ẩn nguy cơ cạn kiệt bộ nhớ. Tuy nhiên, nhờ áp dụng triệt để kỹ thuật chia sẻ cấu trúc Con trỏ cha (Structural Sharing) chỉ lưu 116 bytes cho mỗi nút, dung lượng RAM đỉnh của Greedy trên toàn bộ 9 bài test luôn được kiểm soát an toàn dưới 1.5 MB, loại bỏ hoàn toàn nguy cơ tràn bộ nhớ thường gặp ở các giải thuật tìm kiếm Heuristic truyền thống.



Hình 9: Biểu đồ so sánh bộ nhớ tiêu tốn đỉnh (Peak RAM) giữa DFS và Greedy qua 9 bài kiểm thử.

4.4 Kết luận: Sự đánh đổi giữa Node, Thời gian và Bộ nhớ

Quá trình nghiên cứu và thực nghiệm bài toán *Tents and Trees* làm sáng tỏ mối quan hệ đánh đổi trực tiếp giữa 3 yếu tố cốt lõi trong giải thuật tìm kiếm: số lượng trạng thái duyệt (Node), thời gian thực thi (Thời gian) và dung lượng tài nguyên lưu trữ (Bộ nhớ).

1. Mối quan hệ đánh đổi giữa Số lượng Node và Thời gian thực thi: Thời gian giải quyết bài toán là tích số giữa tổng số node cần duyệt và độ trễ tính toán trên mỗi node. Giải thuật DFS sở hữu chi phí trên mỗi node thấp do chỉ thực hiện các phép gán ô và gọi đệ quy cơ bản, nhờ đó đạt tốc độ vượt trội ở các bài toán nhỏ. Đổi lại, DFS phải chấp nhận sự bùng nổ về số lượng node khi bài toán mở rộng. Ngược lại, Greedy Best-First Search chấp nhận chi phí tính toán Heuristic trên mỗi node cao hơn nhằm đổi lấy khả năng thu hẹp tới hơn 94% số node cần mở rộng. Sự đánh đổi này mang lại lợi thế thời gian rõ rệt cho Greedy ở các bài toán quy mô 15×15, nhưng cũng bộc lộ giới hạn khi chi phí xử lý trên từng nút của môi trường thông dịch bắt đầu lấn át ở bài toán cực lớn 20×20.

2. Mối quan hệ đánh đổi giữa Số lượng Node và Bộ nhớ lưu trữ: Giải thuật DFS hy sinh khả năng ghi nhớ lịch sử trạng thái để đạt mức tiêu thụ bộ nhớ tối thiểu, dẫn đến việc giải thuật có nguy cơ lạc vào các nhánh cụt và phải duyệt lặp lại nhiều trạng thái dư thừa. Trong khi đó, Greedy chủ động cấp phát bộ nhớ để lưu trữ các trạng thái biên trong tập `Open` và nhận diện trạng thái trùng lặp thông qua tập `Closed`. Sự đánh đổi bộ nhớ này đóng vai trò như một bộ lọc thông minh, giúp triệt tiêu hoàn toàn nguy cơ lặp vòng và giữ cho số lượng node mở rộng luôn ở mức tối ưu.

3. Bài toán kinh điển giữa Thời gian và Bộ nhớ (Time-Space Trade-off): Giải thuật DFS lựa chọn phương án an toàn tuyệt đối về bộ nhớ (chỉ vài chục KB) nhưng phải đánh đổi bằng thời gian quay lui kéo dài khi gặp các cấu hình phức tạp. Trái lại, Greedy sử dụng bộ nhớ làm bàn đạp chiến lược nhằm tìm ra con đường ngắn nhất dẫn tới đích. Đóng góp nổi bật của nghiên cứu là nhóm đã tối ưu hóa thành công sự đánh đổi này thông qua cấu trúc Con trỏ cha (Structural Sharing): giải thuật vừa đạt được số node cực thấp, thời gian giải nhanh chóng, mà vẫn khống chế được mức tiêu thụ RAM đỉnh dưới 1.5 MB trên mọi quy mô bài toán.